

Cloud Computing Architecture

Measurements and Plotting

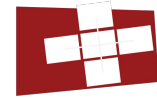
Foteini Strati

foteini.strati@inf.ethz.ch

PhD Student, Systems Group



Agenda



DINFK

1. How to get correct measurements
2. Plotting

Part 1: Measurements



Always measure one level deeper

- Thorough performance evaluation is important in:
 - **Academic research:**
 - Essential for many publications to prove the value of a new idea (prove my system is working)
 - **Industry:**
 - Maintain a high level of performance (across the lifetime) of a product
 - e.g. clients care about SLAs
- **Good performance evaluation:**
 - Deep understanding of a system's behavior - *why does the system behave like that?*
 - Understanding of internal mechanisms and components

[Always Measure One Level Deeper, John Ousterhout, 2018](#)



Always measure one level deeper

- Performance measurement is often done superficially, e.g. just before the paper deadline
- People stop measuring as soon as they find an experiment that gave positive results, omitting the less favorable results

➡ incomplete, misleading, and erroneous measurements



Mistake 1: trusting the numbers

- Performance-measurement code is likely to have bugs, but the bugs are less obvious. These bugs could appear in:
 - The benchmarks themselves
 - The scripts that process/plot the measurements
- **Common misconception:** *If the system is not crashing, the measurements are probably correct*
- Most measurement bugs don't cause crashes; instead they simply produce wrong numbers



Mistake 2: guessing instead of measuring

- It is unsafe to draw conclusions based on intuition without measurements to back them up
- **“What else could it possibly be?”**: not a valid justification
- It is unsafe to present an explanation as fact unless measurements confirm the specific behavior
 - For example, if you think that component A is the bottleneck, measure this and verify!
- **It is important to measure before fixing!**



Mistake 3: superficial measurements

- The **all-to-all** performance of a system is essential:
 - e.g. overall runtime, average latency of a request to the server
 - **However, this is not enough!**
- **We have to measure the “internal” behavior of a system, i.e. per-component**
 - break the performance down (e.g., what components are responsible for which portion of the latency?)
 - what improvement had the greatest impact on performance?



Mistake 4: Confirmation Bias

- Confirmation bias => select and interpret data in a way that support their hypothesis
 - you hope/expect the system will work well
- When you see a result that supports your hypothesis (e.g., “the system has high throughput”), you are more likely to accept the result without question
- In contrast, if a measurement suggests it is not performing well, you are more likely to dig deeper to explain what is happening and fix the problem

➡ results can be inaccurate or misleading



Mistake 5: Not enough time

- **Allow enough time to do performance evaluation!!!**
 - (almost certainly) there will be bugs
 - Many iterations of evaluation/performance improvements
 - Need to evaluate in multiple scenarios/use cases for a thorough analysis



Advice for high-quality measurements

- **Allow lots of time**
- **Don't trust the numbers:**
 - Measurements should be considered *guilty until proven innocent*
 - Take different measurements at the same level
 - Validate with back-of-the-envelope calculations/simulations
- **Use intuition to ask questions, not answer them**
 - Can help identifying directions for further exploration
 - Always validate with data before making decisions or claims
- **Always measure one level deeper**
 - Measure performance of different components
 - Look at the distribution, not only median values



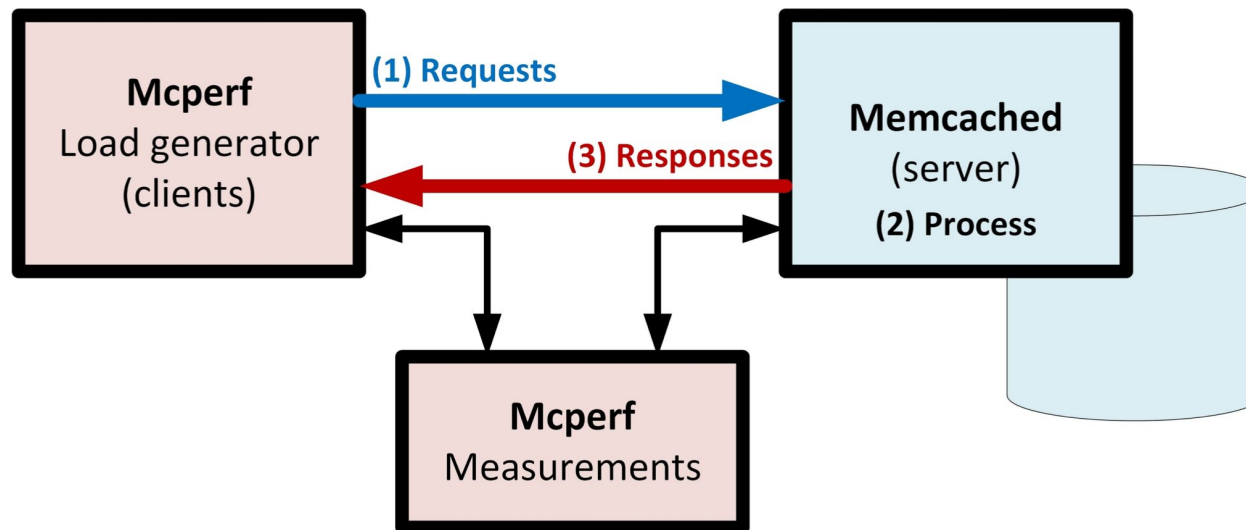
Measurement infrastructure

- **Automate** measurements as much as possible
- Create a **dashboard**
 - a display to show all performance measurements from a particular run
- **Do not remove the instrumentation**
- **Presentation matters**
 - Should expose the correct level of details
 - Easy to understand

Part 2: Plotting



Recap: system under test (part 1)





Testing methodology (part 1)

- Repeat at least 3 times
- **X-axis:** the **actual** achieved **QPS** (not the target)
- **Y-axis:** 95th percentile latency
- **Plot error bars for each point on both axes**
- Report QPS from the measurement machine

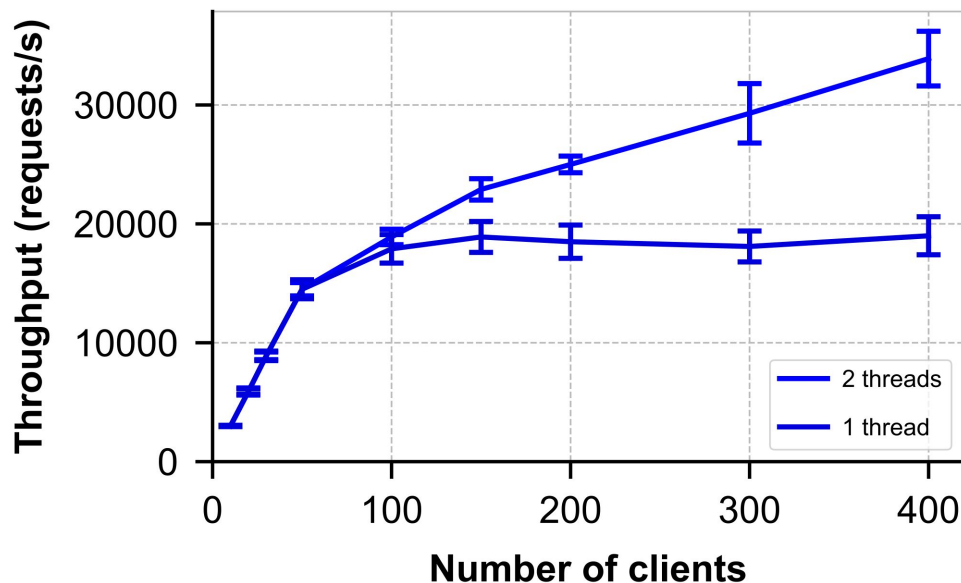
Readability of your plot will be part of your grade!



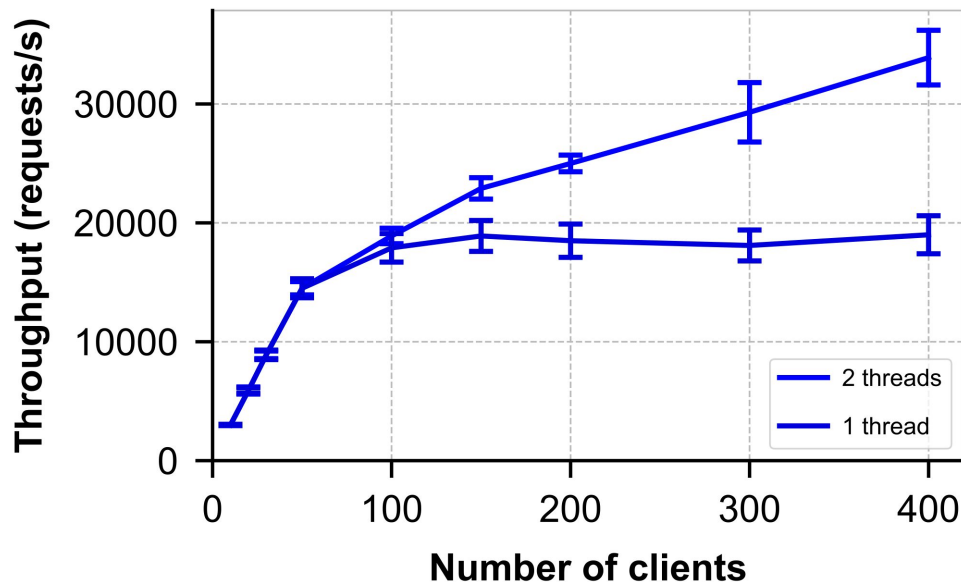
Collect and process results

- **Structure** your measurement data and **store the results of each repetition**:
 - Collect p95 response times of each repetition
 - Collect queries per second (achieved not target!) of each repetition
- Compute average, sample standard deviation
- Use a scripting language that works for you: bash, Python, ...
- Other statistics (e.g., min, max, percentiles, ...) might also be interesting to calculate
- **Automate** as much as possible

Plot 1: What is wrong with this plot?

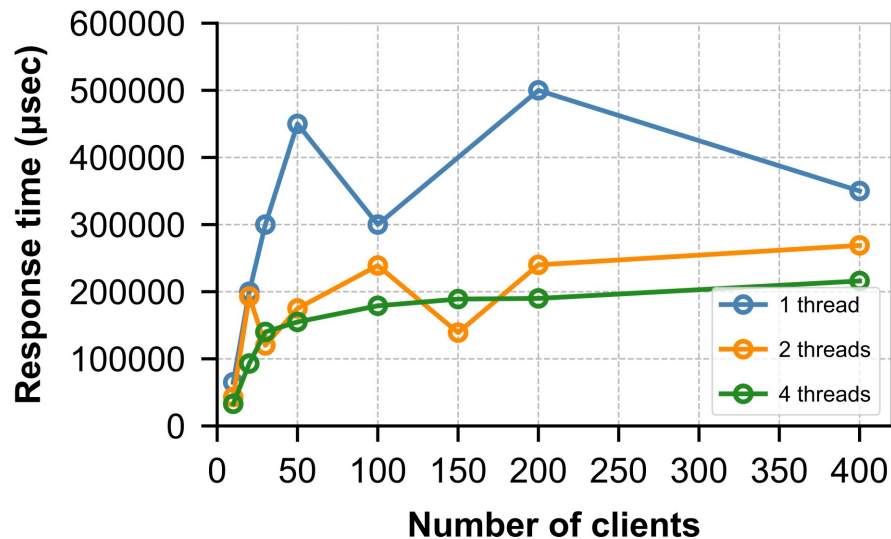


Plot 1: What is wrong with this plot?

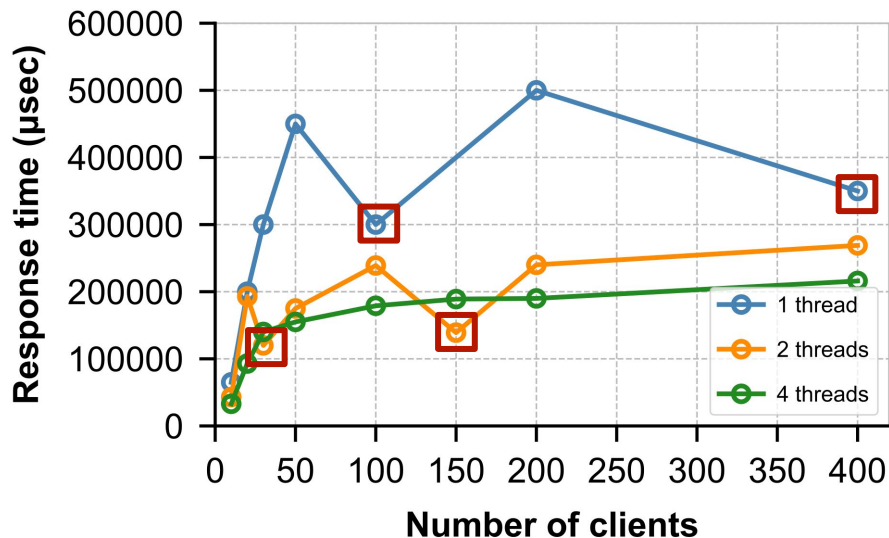


- Misses over-saturated part
- Lines are not distinguishable (bad choice of colors)
- Missing line points

Plot 2: What is wrong with this plot?

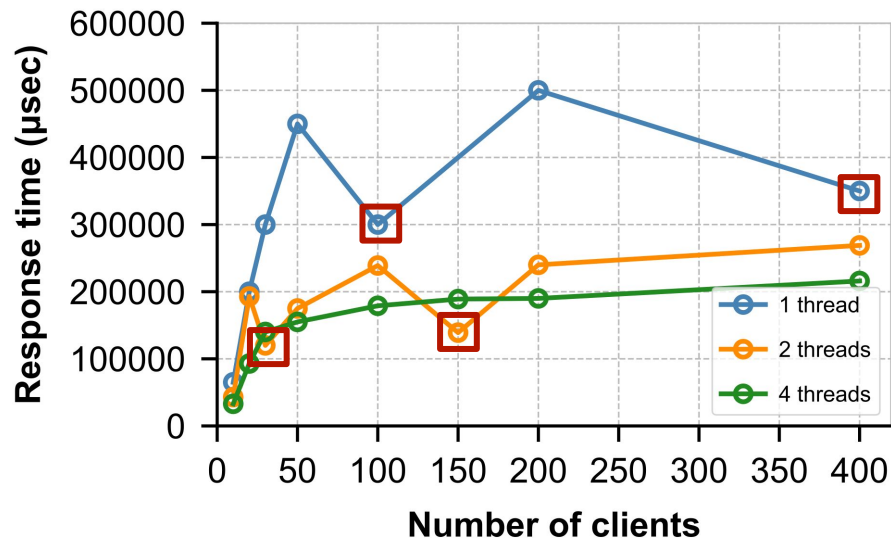


Plot 2: What is wrong with this plot?



- Why these **unexpected dips** in response time?
 - Are requests being lost?
 - Is there a benchmarking problem?
 - Is the same setup used in each experiment?

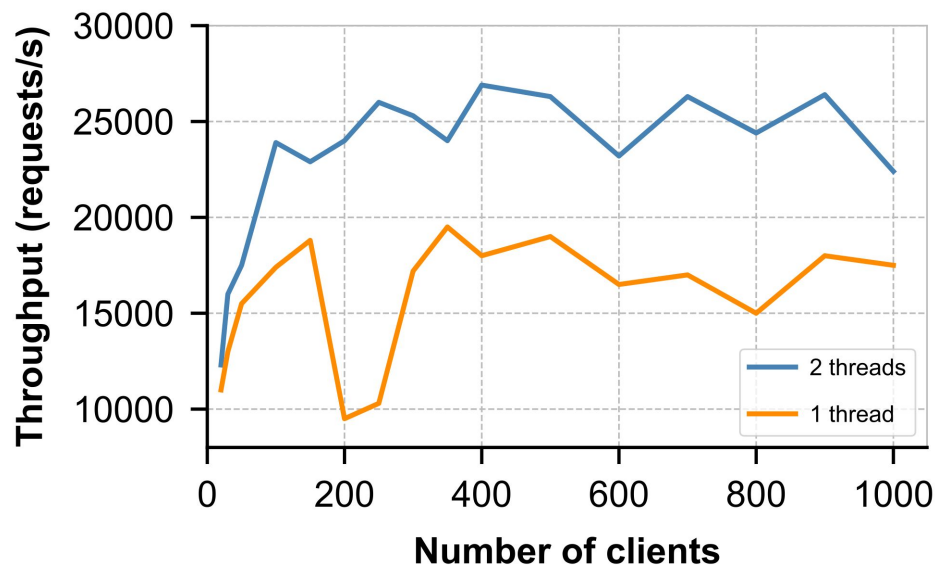
Plot 2: What is wrong with this plot?



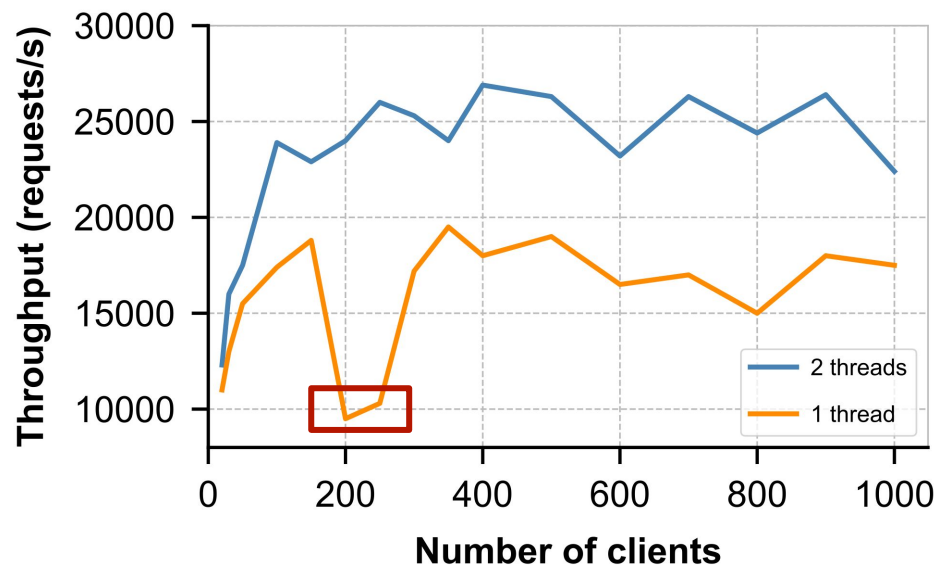
- Why these **unexpected dips** in response time?
 - Are requests being lost?
 - Is there a benchmarking problem?
 - Is the same setup used in each experiment?
- Y-axis would be better in ms-scale
- Error bars absent
- The same symbol used everywhere for the line points



Plot 3: What is wrong with this plot?

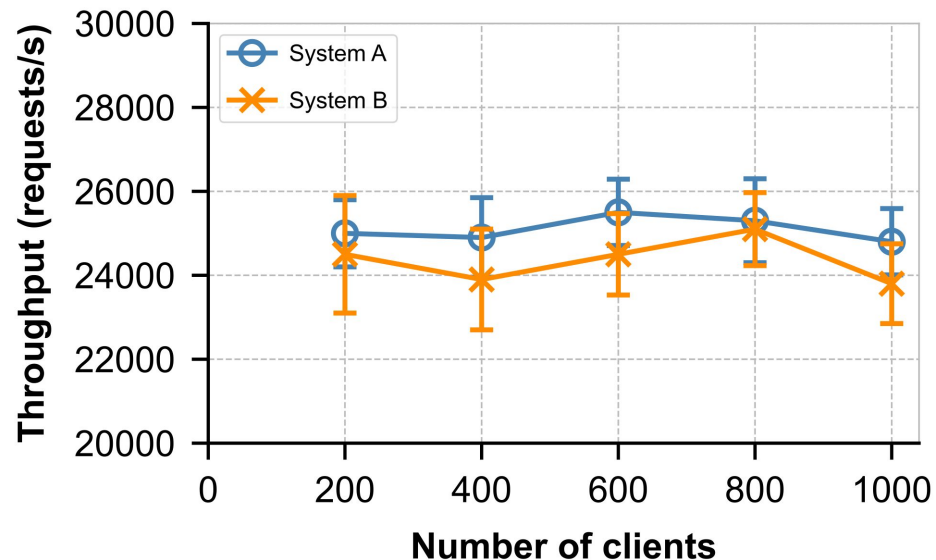


Plot 3: What is wrong with this plot?

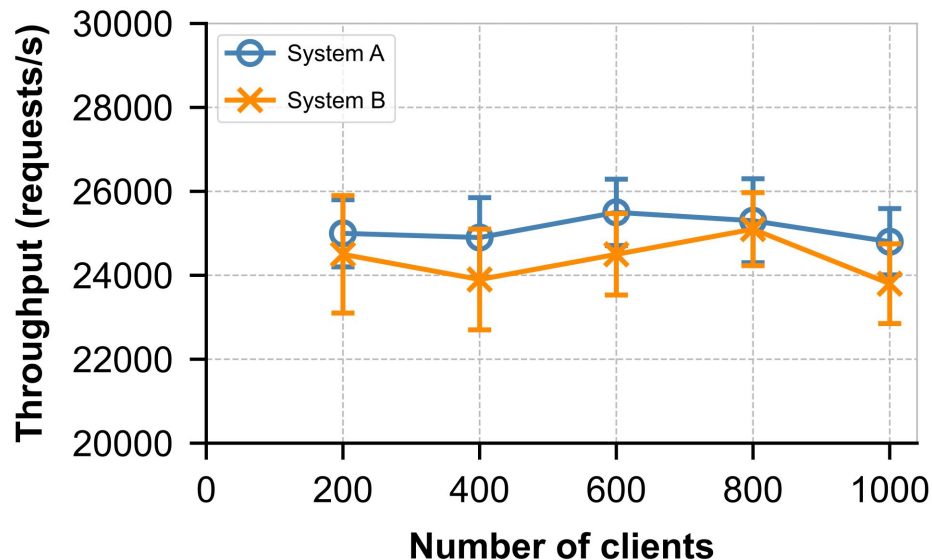


- Y-axis should start from 0
- What is going on **here**?
- Data fluctuations? Were the measurements repeated?
- Error bars and line points are absent

Plot 4: What is wrong with this plot?

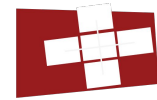


Plot 4: What is wrong with this plot?

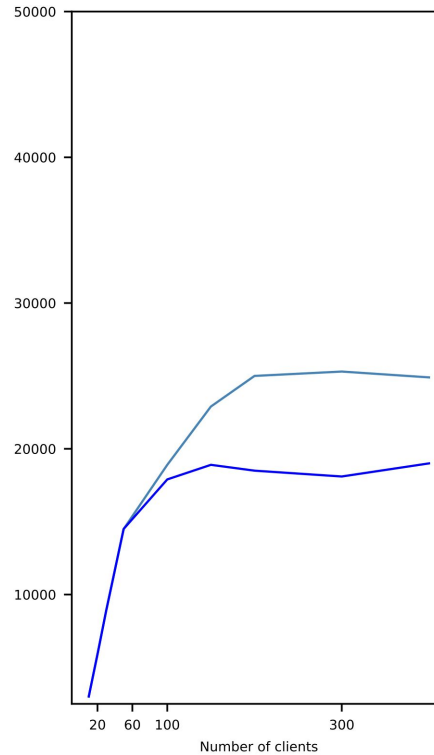


- Y-axis should start from 0
- What happens when:
 - Num_clients < 200?
 - Num_clients > 1000?
- Y-axis would be better in Krequests/s

Find the improvements



DINFK





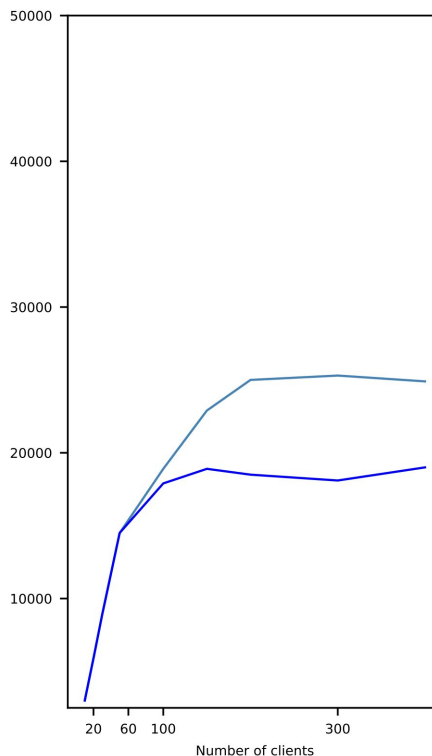
Find the improvements

Y-axis:

- Missing label
- Does not start at 0
- Upper limit too high

Inconsistent ticks at X axis

Font size too small



No legend

Line colors not distinguishable

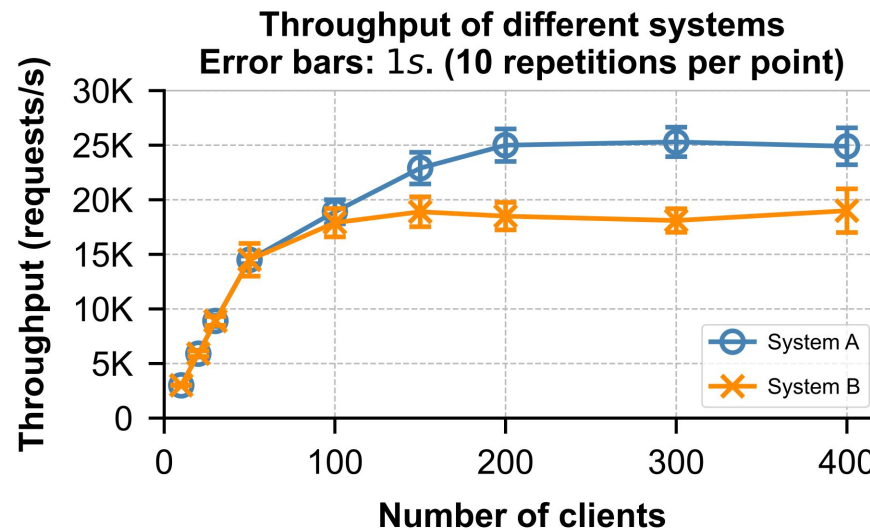
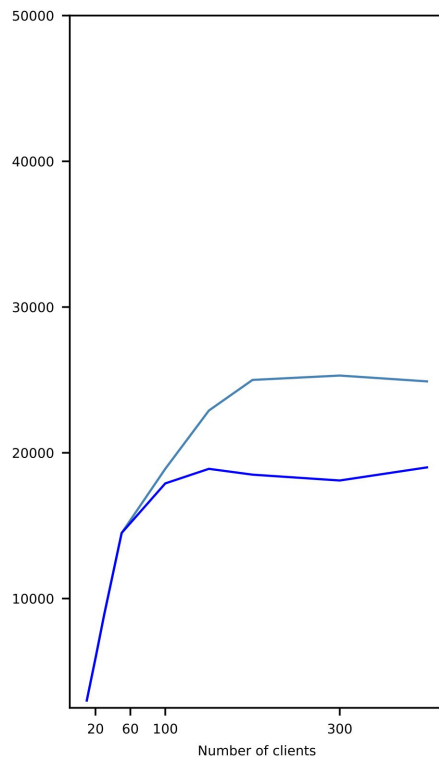
Missing line points

Missing error bars

After the improvements



DINFK





Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached.



Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**



Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at...



Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at... **DESCRIPTION**

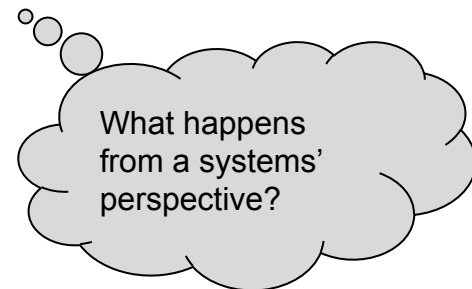


Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at... **DESCRIPTION**
- **Example sentence 3:** The throughput saturates at X number of clients, because...

Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at... **DESCRIPTION**
- **Example sentence 3:** The throughput saturates at X number of clients, because... **EXPLANATION**





Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at... **DESCRIPTION**
- **Example sentence 3:** The throughput saturates at X number of clients, because... **EXPLANATION**
- **Example sentence 4:** The interactive law holds as expected, as shown in Figure 1.



Description vs Explanation

- **Example sentence 1:** We observe in Figure 1 that the throughput increases linearly with the number of clients, until X number of clients is reached. **DESCRIPTION**
- **Example sentence 2:** Since the rate of increase in response time changes suddenly at X number of clients, we deduce the system is saturated at... **DESCRIPTION**
- **Example sentence 3:** The throughput saturates at X number of clients, because... **EXPLANATION**
- **Example sentence 4:** The interactive law holds as expected, as shown in Figure 1. **DESCRIPTION**



Plotting - best practices

- **Start axis at zero**, and keep same range for related graphs
- **Label both axes, state units clearly:**
 - Use understandable units: e.g., requests/s, not requests/minute
 - Instead of 1200000 use 1.2M or 1.2 million
 - **Caution with logarithmic scales on axis**
- **Include error bars! AND: explain** what the error bars represent!
- Make sure system configuration is easily found: what was measured?



Benchmarking - recommendations

- As you progress and rerun experiments save results in different files:
 - you can regenerate graphs for different versions of the result
- Keep style consistent over graphs
- **Automate and use scripts for exporting graphs**
- **Plotting tools:**
 - gnuplot (shell)
 - matplotlib (Python)
 - pgfplots (LaTeX)
 - Excel (Windows) (though, not (easily) automated)
- **No hand-drawn plots!**

Example: Matplotlib

Create a plot

- `import matplotlib.pyplot as plt`
- `fig = plt.figure()`
- `fig_ax = fig.gca()`
- ...
- `plt.tight_layout()`
- `plt.savefig("example.pdf")`

Plot data

- `fig_ax.plot(x, y,...)`
- `fig_ax.errorbar(x,y,xerr=...,yerr=...,...)`

Fix layout

- `fig_ax.set_xlabel(...), fig_ax.set_ylabel(...)`
- `fig_ax.grid(...)`
- `fig_ax.set_xlim(...), fig_ax.set_ylim(...)`
- `fig_ax.tick_params(...)`
- `fig_ax.set_xticks(...), fig_ax.set_yticks(...)`
- `fig_ax.set_xticklabels(...), fig_ax.set_yticklabels(...)`
- `fig_ax.legend(...)`



Thank you!
Questions?